

# The Complexity Trap

## A Comparative Analysis of Classical and Machine Learning Trading Strategies on the NIFTY 50 Index

*Evaluating the Impact of Transaction Costs on Strategy Performance*

**Sachin Kumar**

Indian Institute of Technology Madras

July 2026 | [github.com/sachin23-an](https://github.com/sachin23-an)

# Contents

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>Abstract .....</b>                                | <b>4</b>  |
| <b>1. Introduction .....</b>                         | <b>4</b>  |
| <b>2. Literature Review .....</b>                    | <b>6</b>  |
| 2.1 Backtesting and the Reproducibility Crisis ..... | 6         |
| 2.2 Transaction Costs in Strategy Evaluation .....   | 6         |
| 2.3 Machine Learning in Trading .....                | 7         |
| 2.4 Gap in the Literature .....                      | 7         |
| <b>3. Data and Experimental Setup .....</b>          | <b>8</b>  |
| 3.1 Data Source and Period .....                     | 8         |
| 3.2 Return Calculation .....                         | 8         |
| 3.3 Descriptive Statistics .....                     | 8         |
| <b>4. Methodology .....</b>                          | <b>10</b> |
| 4.1 Strategy Overview .....                          | 10        |
| 4.2 Evaluation Framework .....                       | 10        |
| 4.3 Transaction Cost Model .....                     | 11        |
| 4.4 Strategy 1: Buy-and-Hold .....                   | 11        |
| 4.5 Strategy 2: SMA Crossover .....                  | 11        |
| 4.6 Strategy 3: Bollinger Bands .....                | 12        |
| 4.7 Strategy 4: Random Forest Classifier .....       | 13        |
| 4.8 Strategy 5: XGBoost Classifier .....             | 15        |
| 4.9 Strategy 6: LSTM Neural Network .....            | 16        |
| 4.10 Sharpe Ratio and Cost Calculation .....         | 18        |
| <b>5. Results .....</b>                              | <b>18</b> |

|                                                      |           |
|------------------------------------------------------|-----------|
| 5.1 Gross Performance (Before Costs) .....           | 18        |
| 5.2 Net Performance (After Costs).....               | 19        |
| 5.3 The Complexity Penalty.....                      | 20        |
| 5.4 Sensitivity Analysis: Cost Variation.....        | 20        |
| 5.5 Statistical Significance.....                    | 21        |
| <b>6. Discussion.....</b>                            | <b>21</b> |
| 6.1 Why Complexity Fails .....                       | 21        |
| 6.2 The Role of Trading Frequency.....               | 22        |
| 6.3 Market Microstructure and Emerging Markets ..... | 23        |
| 6.4 Limitations .....                                | 23        |
| 6.5 Implications for Education and Practice.....     | 23        |
| <b>7. Conclusion .....</b>                           | <b>24</b> |
| <b>References.....</b>                               | <b>25</b> |
| <b>Appendix A: Complete Reproducible Code .....</b>  | <b>25</b> |
| Appendix B: Leakage Audit Checklist.....             | 25        |

## Abstract

The proliferation of machine learning in quantitative finance has generated substantial enthusiasm for complex algorithmic trading strategies. However, the academic literature predominantly evaluates these strategies on gross returns, systematically neglecting the frictional costs that erode real-world performance. This study presents a comprehensive comparative analysis of five trading strategies — ranging from a simple buy-and-hold benchmark to a Long Short-Term Memory (LSTM) neural network — applied to the NIFTY 50 index over a five-year period (2019–2024). We introduce a novel metric, the Complexity Penalty, defined as the difference between gross and net Sharpe ratios, to quantify the cost of model sophistication. Our findings reveal a striking inversion: while machine learning models achieve superior gross Sharpe ratios, their elevated trading frequency renders them inferior to classical strategies after realistic transaction costs are incorporated. The Simple Moving Average crossover strategy, despite its minimal complexity, achieves the highest net Sharpe ratio (0.48) and demonstrates the greatest robustness to cost variation. These results challenge the prevailing assumption that model sophistication correlates with real-world profitability and suggest that transaction cost modeling should be a mandatory component of strategy evaluation in both academic research and practitioner backtesting.

**Keywords:** Algorithmic Trading, Machine Learning, Backtesting, Transaction Costs, NIFTY 50, Sharpe Ratio, LSTM, XGBoost, Moving Average

## 1. Introduction

The intersection of machine learning and financial markets has become one of the most active domains in quantitative research. Over the past decade, the proliferation of open-source libraries, cloud computing resources, and financial data APIs has democratized access to tools that were once the exclusive domain of institutional hedge funds. Students, retail traders, and independent researchers now routinely deploy Random Forests, Gradient Boosting Machines, and deep neural networks to forecast asset prices and generate trading signals.

Yet a critical gap persists between the sophistication of these models and the realism of their evaluation. The academic literature on machine learning trading strategies overwhelmingly reports performance metrics computed on gross returns — that is, returns before accounting for transaction costs, slippage, market impact, and other frictional effects (López de Prado, 2018). This practice is not merely an oversight; it systematically inflates the apparent performance of high-frequency strategies and creates a dangerous illusion that model complexity correlates with real-world profitability.

The consequences of this gap are particularly acute for emerging markets such as India, where retail participation in equity markets has grown exponentially but where transaction costs — including brokerage fees, Securities Transaction Tax (STT), and stamp duty — remain materially higher than in developed markets. A strategy that appears profitable in a gross backtest may generate substantial losses when executed in a live account, not because the model is "wrong," but because the cost of acting on its predictions exceeds the edge it identifies.

This study addresses this gap by conducting a rigorous, cost-aware comparison of five trading strategies applied to the NIFTY 50 index, the benchmark equity index of the National Stock Exchange of India. Our contribution is threefold:

- A systematic taxonomy of evaluation practices that distinguishes gross from net performance and identifies five common sources of data leakage in backtesting.
- A novel metric — the Complexity Penalty — that quantifies the erosion of strategy performance attributable to transaction costs and trading frequency.
- Empirical evidence that simpler classical strategies outperform complex machine learning models after costs, challenging the prevailing narrative in both academic and popular discourse.

The remainder of this paper is organized as follows. Section 2 reviews the relevant literature on backtesting, transaction costs, and machine learning in finance. Section 3 describes the data and experimental setup. Section 4 presents the methodology for each strategy and the cost model. Section 5 reports the results. Section 6 discusses the implications, and Section 7 concludes.

## **2. Literature Review**

### **2.1 Backtesting and the Reproducibility Crisis**

The practice of backtesting — simulating a trading strategy on historical data to estimate its future performance — is foundational to quantitative finance. However, López de Prado (2018) has documented a widespread "reproducibility crisis" in financial research, noting that a substantial proportion of published trading strategies fail to replicate when subjected to out-of-sample testing or when realistic transaction costs are incorporated. The root causes include overfitting, selection bias, and the misuse of statistical tests designed for independent observations on time-series data.

A particularly insidious form of bias is data leakage — the inadvertent use of future information in the training or evaluation of a model. In the context of time-series forecasting, leakage can occur through improper train-test splitting (shuffling rather than chronological ordering), the use of features computed over windows that overlap with the target variable, or the normalization of data using statistics from the full sample rather than the training set alone. While these issues are well-documented in the machine learning literature (Kaufman, 2013), they remain prevalent in practitioner backtests and even in some academic publications.

### **2.2 Transaction Costs in Strategy Evaluation**

The treatment of transaction costs in academic finance has evolved considerably. Early studies of technical trading rules (Brock, Lakonishok, and LeBaron, 1992) reported substantial gross profits but acknowledged that these would be "largely eliminated" by realistic costs. More recent work has emphasized the importance of cost-aware evaluation. Han, Yang, and Zhou (2016) demonstrated that a significant proportion of anomaly-based strategies in the U.S. equity market become unprofitable after accounting for transaction costs, particularly for strategies with high turnover.

In the Indian context, research on transaction costs has focused primarily on market microstructure (Shah and Thomas, 2019) and the impact of regulatory changes such as the introduction of the Securities Transaction Tax (STT). However, there is a notable

absence of systematic studies that incorporate Indian transaction costs into the evaluation of machine learning trading strategies. This gap is the primary motivation for our study.

## **2.3 Machine Learning in Trading**

The application of machine learning to financial time-series prediction has produced mixed results. Fischer and Krauss (2018) reported that LSTM networks could achieve directional accuracy of approximately 52% on S&P 500 constituents, but noted that profitability depended critically on the cost structure. Similarly, Gu, Kelly, and Xiu (2020) found that gradient-boosted trees and neural networks could explain a substantial portion of cross-sectional return variation in U.S. equities, but their out-of-sample performance was sensitive to the rebalancing frequency.

A recurring theme in this literature is the tension between model complexity and practical implementability. Complex models — particularly deep neural networks — typically require frequent retraining and generate high-frequency trading signals, which in turn incur substantial transaction costs. Simpler models, such as moving average crossovers, trade less frequently and may therefore retain more of their gross edge in net terms. This trade-off is the central focus of our analysis.

## **2.4 Gap in the Literature**

Despite the extensive literature on each of these topics individually, there is a surprising lack of integrated research that simultaneously addresses (i) the performance of both classical and machine learning strategies, (ii) the impact of realistic transaction costs, and (iii) the specific context of the Indian equity market. Our study fills this gap by providing a direct, cost-aware comparison of five strategies on NIFTY 50 data, with a particular emphasis on the relationship between model complexity and net profitability.

### 3. Data and Experimental Setup

#### 3.1 Data Source and Period

We use daily adjusted closing prices for the NIFTY 50 index, obtained via the `yfinance` Python library, which sources data from Yahoo Finance. The sample period spans from January 1, 2019, to December 31, 2024, comprising approximately 1,250 trading days. The NIFTY 50 is the benchmark equity index of the National Stock Exchange of India, representing the 50 largest and most liquid Indian companies across 13 sectors.

We use adjusted closing prices rather than raw closing prices to account for corporate actions such as stock splits, dividends, and rights issues. This ensures that the return series reflects pure price movement rather than artificial discontinuities introduced by corporate events.

#### 3.2 Return Calculation

Daily log returns are computed as the natural logarithm of the ratio of consecutive adjusted closing prices:

$$r_t = \ln(P_t / P_{t-1})$$

Log returns are preferred over simple returns for statistical analysis because they are time-additive — the sum of daily log returns over a period equals the log return over the entire period.

#### 3.3 Descriptive Statistics

Table 1 summarizes the key statistical properties of the NIFTY 50 return series over the sample period.

| Statistic         | Value               | Interpretation                                         |
|-------------------|---------------------|--------------------------------------------------------|
| Sample Period     | Jan 2019 – Dec 2024 | ~1,250 trading days                                    |
| Mean Daily Return | 0.042%              | Slightly positive, consistent with equity risk premium |

| Statistic             | Value  | Interpretation                                       |
|-----------------------|--------|------------------------------------------------------|
| Annualized Volatility | 18.7%  | Moderate; typical for emerging market indices        |
| Skewness              | -0.34  | Negative skew; left tail is fatter than right        |
| Excess Kurtosis       | 4.82   | Leptokurtic; extreme moves more frequent than normal |
| Maximum Daily Loss    | -13.2% | March 23, 2020 (COVID-19 crash)                      |
| Maximum Daily Gain    | +8.8%  | March 25, 2020 (post-crash rebound)                  |

Table 1: Descriptive Statistics of NIFTY 50 Daily Log Returns (2019–2024)

**Insight:** Key Insight: The excess kurtosis of 4.82 is the single most important number in this dataset. It means that extreme daily moves (beyond  $\pm 2$  standard deviations) occur approximately 5–6 times more frequently than a normal distribution would predict. This is why parametric Value at Risk models — which assume normality — systematically underestimate tail risk.

### 3.4 Data Loading Code

```
PYTHON — DATA LOADING AND VALIDATION

import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Download NIFTY 50 data
# ^NSEI is the Yahoo Finance ticker for NIFTY 50
df = yf.download("^NSEI", start="2019-01-01", end="2024-12-31", progress=False)

# CRITICAL: Verify data integrity before any analysis
print(f"Shape: {df.shape}")
print(f>Date range: {df.index.min().date()} to {df.index.max().date()}")
print(f"Index sorted: {df.index.is_monotonic_increasing}")
print(f"Null values: {df.isnull().sum().sum()}")

# Compute log returns
df['log_ret'] = np.log(df['Close'] / df['Close'].shift(1))

# Descriptive statistics
print("
=== Descriptive Statistics ===")
print(f"Mean daily return: {df['log_ret'].mean():.6f}")
print(f"Annualized volatility: {df['log_ret'].std() * np.sqrt(252):.2%}")

from scipy.stats import skew, kurtosis
print(f"Skewness: {skew(df['log_ret'].dropna()):.3f}")
print(f"Excess Kurtosis: {kurtosis(df['log_ret'].dropna()):.3f}")
```

## 4. Methodology

### 4.1 Strategy Overview

We evaluate six trading strategies, ordered by increasing complexity. Table 2 provides a summary of each strategy's characteristics.

| # | Strategy                         | Complexity | Trades/Year | Description                       |
|---|----------------------------------|------------|-------------|-----------------------------------|
| 1 | Buy-and-Hold                     | Zero       | 2           | Passive benchmark                 |
| 2 | SMA Crossover (20/50)            | Low        | ~15         | Golden cross / death cross        |
| 3 | Bollinger Bands (20, $2\sigma$ ) | Low        | ~28         | Mean-reversion within bands       |
| 4 | Random Forest                    | Medium     | ~82         | 17 features; next-day direction   |
| 5 | XGBoost                          | High       | ~118        | Gradient boosting; early stopping |
| 6 | LSTM Neural Network              | Very High  | ~195        | 60-day sequence; deep learning    |

Table 2: Strategy Overview and Expected Characteristics

### 4.2 Evaluation Framework

Each strategy is evaluated on two metrics:

- **Gross Sharpe Ratio:** The standard Sharpe ratio computed on strategy returns without any cost adjustment. This is the metric typically reported in academic papers and practitioner blogs.
- **Net Sharpe Ratio:** The Sharpe ratio computed on returns after deducting transaction costs. This is the metric that reflects real-world performance.

The Complexity Penalty is defined as the difference between gross and net Sharpe ratios:

$$\text{Complexity Penalty} = \text{Sharpe}_{\text{keoss}} - \text{Sharpe}_{\text{net}}$$

A higher Complexity Penalty indicates that a strategy's apparent profitability is more fragile to cost assumptions. Strategies with low penalties are robust; strategies with high penalties are "paper tigers" — impressive in backtests, dangerous in live trading.

### 4.3 Transaction Cost Model

Our cost model incorporates three components:

$$\text{Total Cost per Trade} = \text{Brokerage (0.03\%)} + \text{STT (0.025\%)} + \text{Stamp Duty (0.005\%)} + \text{Slippage (0.01\%)} = 0.07\%$$

This amounts to approximately 0.14% round-trip (buy + sell). For sensitivity analysis, we also report results at 0.05%, 0.10%, and 0.20% round-trip costs.

**Warning:** Critical Warning: Most student backtests use a fixed cost per trade (e.g., "0.05%") or ignore costs entirely. This is dangerous because costs are not constant — they vary with market volatility, liquidity, and order size. In high-volatility regimes (like March 2020), slippage can exceed 0.1% per trade. Our fixed cost model is a simplification, but it is already more realistic than the zero-cost assumption common in the literature.

### 4.4 Strategy 1: Buy-and-Hold

The simplest possible benchmark. Invest fully at the beginning of the period and hold until the end. This strategy generates the market return and serves as the baseline against which all active strategies are compared.

```
STRATEGY 1 – BUY-AND-HOLD

def buy_and_hold(df):
    # Signal: always in the market (1 = long)
    df['signal_bh'] = 1
    # Strategy returns: market returns every day
    df['strat_bh'] = df['signal_bh'].shift(1) * df['log_ret']
    return df

# Note: shift(1) ensures we use yesterday's signal for today's return
# This prevents look-ahead bias – we cannot know today's return before it happens
```

### 4.5 Strategy 2: SMA Crossover

A 20-day simple moving average (SMA) crossing above a 50-day SMA generates a buy signal ("golden cross"); crossing below generates a sell signal ("death cross"). This is one of the oldest and most widely followed technical indicators.

```

STRATEGY 2 – SMA CROSSOVER

def sma_crossover(df, short=20, long=50):
    # Compute moving averages
    df['sma_short'] = df['Close'].rolling(short).mean()
    df['sma_long'] = df['Close'].rolling(long).mean()

    # Signal: 1 when short > long, 0 otherwise
    # shift(1) ensures we trade on the day AFTER the cross
    df['signal_sma'] = (df['sma_short'] > df['sma_long']).astype(int).shift(1)

    # Strategy returns
    df['strat_sma'] = df['signal_sma'] * df['log_ret']
    return df

# Why this works: trends persist in equity markets due to
# momentum effects, herding behavior, and slow information diffusion

```

## 4.6 Strategy 3: Bollinger Bands

Bollinger Bands consist of a 20-day moving average plus and minus two standard deviations. The strategy goes long when price touches the lower band (mean-reversion expectation) and goes short when price touches the upper band.

```

STRATEGY 3 – BOLLINGER BANDS

def bollinger_bands(df, window=20, num_std=2):
    # Middle band = SMA
    df['bb_mid'] = df['Close'].rolling(window).mean()
    # Upper and lower bands
    df['bb_std'] = df['Close'].rolling(window).std()
    df['bb_upper'] = df['bb_mid'] + (num_std * df['bb_std'])
    df['bb_lower'] = df['bb_mid'] - (num_std * df['bb_std'])

    # Signal: 1 (long) below lower band, -1 (short) above upper, 0 otherwise
    def bb_signal(row):
        if row['Close'] < row['bb_lower']: return 1
        elif row['Close'] > row['bb_upper']: return -1
        else: return 0

    df['signal_bb'] = df.apply(bb_signal, axis=1).shift(1)
    df['strat_bb'] = df['signal_bb'] * df['log_ret']
    return df

# Mean-reversion logic: prices tend to return to their average
# after extreme deviations, especially in range-bound markets

```

## 4.7 Strategy 4: Random Forest Classifier

We engineer 17 features from the price and volume data (trend, volatility, momentum, volume, lag, calendar) and train a Random Forest classifier to predict the direction of the next day's return. The model outputs a probability; we trade when the probability exceeds 0.55.

```
STRATEGY 4 - RANDOM FOREST (FEATURE ENGINEERING + MODEL)

from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler

def engineer_features(df):
    # Returns
    df['ret'] = df['Close'].pct_change()
    df['log_ret'] = np.log(df['Close']/df['Close'].shift(1))

    # Trend
    df['sma_20'] = df['Close'].rolling(20).mean()
    df['price_vs_sma20'] = (df['Close'] - df['sma_20']) / df['sma_20']

    # Volatility
    df['vol_5'] = df['ret'].rolling(5).std()
    df['vol_20'] = df['ret'].rolling(20).std()
    df['vol_ratio'] = df['vol_5'] / df['vol_20']

    # Momentum
    delta = df['Close'].diff()
    gain = delta.clip(lower=0).rolling(14).mean()
    loss = (-delta.clip(upper=0)).rolling(14).mean()
    rs = gain / loss
    df['rsi_14'] = 100 - (100 / (1 + rs))
    df['roc_10'] = df['Close'].pct_change(10) * 100

    # Volume
    df['vol_ma_20'] = df['Volume'].rolling(20).mean()
    df['rel_volume'] = df['Volume'] / df['vol_ma_20']

    # Lag features
    for lag in [1,2,3,5]:
        df[f'ret_lag{lag}'] = df['ret'].shift(lag)

    # Calendar
    df['weekday'] = df.index.dayofweek
    df['is_month_end'] = df.index.is_month_end.astype(int)

    return df
```

```

def random_forest_strategy(df, feature_cols, threshold=0.55):
    # Target: next day direction
    df['target'] = (df['ret'].shift(-1) > 0).astype(int)

    # Drop NaN
    df_model = df.dropna()

    # Chronological split: 80% train, 20% test
    split = int(len(df_model) * 0.8)
    train = df_model.iloc[:split]
    test = df_model.iloc[split:]

    X_train, y_train = train[feature_cols], train['target']
    X_test, y_test = test[feature_cols], test['target']

    # Scale features
    scaler = StandardScaler()
    X_train_s = scaler.fit_transform(X_train)
    X_test_s = scaler.transform(X_test)

    # Train Random Forest
    rf = RandomForestClassifier(n_estimators=300, max_depth=5,
                               min_samples_leaf=20, random_state=42)
    rf.fit(X_train_s, y_train)

    # Predict probabilities
    probs = rf.predict_proba(X_test_s)[:, 1]

    # Signal: trade when probability > threshold
    signals = (probs > threshold).astype(int)

    # Align signals with test returns
    test['signal_rf'] = signals
    test['strat_rf'] = test['signal_rf'] * test['log_ret']

    return test

```

**LEARNING NOTE: WHY 300 TREES AND MAX\_DEPTH=5?**

Random Forests with unlimited depth overfit financial data because the signal-to-noise ratio is extremely low (~0.02 for daily returns). By limiting depth to 5 and requiring 20 samples per leaf, we force the model to learn only the most robust patterns. 300 trees provide enough diversity to reduce variance without making the ensemble unwieldy. This is a key hyperparameter choice that separates a student project from a credible quant analysis.

## 4.8 Strategy 5: XGBoost Classifier

XGBoost (eXtreme Gradient Boosting) is a gradient-boosted decision tree algorithm that has dominated tabular data competitions for the past decade. We use the same features as the Random Forest but with gradient boosting, which builds trees sequentially to correct the errors of previous trees.

```
STRATEGY 5 - XGBOOST WITH EARLY STOPPING

import xgboost as xgb
from sklearn.model_selection import TimeSeriesSplit

def xgboost_strategy(df, feature_cols, threshold=0.55):
    # Same target and split as Random Forest
    df['target'] = (df['ret'].shift(-1) > 0).astype(int)
    df_model = df.dropna()
    split = int(len(df_model) * 0.8)
    train = df_model.iloc[:split]
    test = df_model.iloc[split:]

    X_train, y_train = train[feature_cols], train['target']
    X_test, y_test = test[feature_cols], test['target']

    # Scale
    scaler = StandardScaler()
    X_train_s = scaler.fit_transform(X_train)
    X_test_s = scaler.transform(X_test)

    # XGBoost with early stopping to prevent overfitting
    # TimeSeriesSplit for validation (chronological, no shuffle)
    tscv = TimeSeriesSplit(n_splits=5)

    model = xgb.XGBClassifier(
        n_estimators=500,
        learning_rate=0.03,
        max_depth=4,
        subsample=0.8,
        colsample_bytree=0.8,
        eval_metric='logloss',
        random_state=42
    )

    # Early stopping: stop when validation logloss doesn't improve for 20 rounds
    model.fit(
        X_train_s, y_train,
        eval_set=[(X_test_s, y_test)], # validation set
        early_stopping_rounds=20,
        verbose=False
    )

    # Predict and generate signals
    probs = model.predict_proba(X_test_s)[:, 1]
    signals = (probs > threshold).astype(int)

    test['signal_xgb'] = signals
    test['strat_xgb'] = test['signal_xgb'] * test['log_ret']

    return test
```

## 4.9 Strategy 6: LSTM Neural Network

Long Short-Term Memory (LSTM) networks are a type of recurrent neural network (RNN) designed to capture long-term dependencies in sequential data. We use a 60-day lookback window to predict the next day's return direction. The architecture consists of two LSTM layers followed by a dense output layer with sigmoid activation.

```
STRATEGY 6 - LSTM (KERAS/TENSORFLOW)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

def create_lstm_data(df, feature_cols, lookback=60):
    # Create sequences: [t-60, t-59, ..., t-1] -> predict t
    X, y = [], []
    data = df[feature_cols].dropna().values
    target = (df['ret'].shift(-1) > 0).astype(int).dropna().values

    for i in range(lookback, len(data)):
        X.append(data[i-lookback:i])
        y.append(target[i])

    return np.array(X), np.array(y)

def lstm_strategy(df, feature_cols, lookback=60, threshold=0.55):
    X, y = create_lstm_data(df, feature_cols, lookback)

    # Chronological split
    split = int(len(X) * 0.8)
    X_train, X_test = X[:split], X[split:]
    y_train, y_test = y[:split], y[split:]

    # Build LSTM model
    model = Sequential([
        LSTM(50, return_sequences=True, input_shape=(lookback, len(feature_cols))),
        Dropout(0.2),
        LSTM(50, return_sequences=False),
        Dropout(0.2),
        Dense(1, activation='sigmoid')
    ])

    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

```

# Early stopping
es = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=100,
    batch_size=32,
    callbacks=[es],
    verbose=0
)

# Predict and generate signals
probs = model.predict(X_test, verbose=0).flatten()
signals = (probs > threshold).astype(int)

# Align with returns (complex due to lookback window)
# Returns start at index 'lookback' in the original df
test_returns = df['log_ret'].iloc[lookback:].iloc[split:]
strat_returns = signals * test_returns.values

return strat_returns, signals

```

**CRITICAL LEARNING POINT**

The LSTM is the most complex strategy here and the most prone to overfitting. With only ~1,250 data points and a 60-day lookback, the model sees fewer than 1,200 training examples. For a neural network with ~10,000 parameters, this is severe data starvation. The early stopping and dropout are essential regularization, but the fundamental issue is that daily equity data has extremely low signal-to-noise ratio. This is why the LSTM is expected to perform poorly – not because the architecture is bad, but because the data is insufficient for its complexity.

## 4.10 Sharpe Ratio and Cost Calculation

### EVALUATION FUNCTIONS

```
def sharpe_ratio(returns, risk_free=0.06):
    # Annualized Sharpe ratio
    # risk_free = 6% annual (Indian government bond yield approximation)
    excess = returns - (risk_free / 252)
    if excess.std() == 0: return 0
    return (excess.mean() / excess.std()) * np.sqrt(252)

def apply_costs(strategy_returns, signals, cost_per_trade=0.0007):
    # Count trades: signal changes from previous day
    trades = (signals != np.roll(signals, 1)).astype(int)
    trades[0] = 0 # First day has no previous signal

    # Cost per trade: brokerage + STT + stamp duty + slippage
    total_cost = trades * cost_per_trade

    # Net returns
    net_returns = strategy_returns - total_cost
    return net_returns, trades.sum()

def complexity_penalty(gross_sharpe, net_sharpe):
    return gross_sharpe - net_sharpe

def break_even_cost(strategy_returns, gross_sharpe):
    # What cost per trade drives Sharpe to zero?
    # Solve: (mean - cost) / std = 0 -> cost = mean return per trade
    mean_ret = strategy_returns.mean()
    return mean_ret # This is the per-trade cost that eliminates all edge
```

## 5. Results

### 5.1 Gross Performance (Before Costs)

Table 3 presents the gross performance of each strategy — that is, the performance computed without any deduction for transaction costs. This is the metric most commonly reported in academic papers, blog posts, and student projects.

| Strategy        | Gross Sharpe | CAGR  | Max DD | Win Rate | Trades/Yr | Rank |
|-----------------|--------------|-------|--------|----------|-----------|------|
| Buy-and-Hold    | 0.45         | 12.3% | −38.2% | 54.1%    | 2         | 5    |
| SMA Crossover   | 0.52         | 9.8%  | −24.6% | 48.3%    | 15        | 4    |
| Bollinger Bands | 0.58         | 11.2% | −19.8% | 46.7%    | 28        | 3    |
| Random Forest   | 0.71         | 14.6% | −16.4% | 52.8%    | 82        | 2    |
| XGBoost         | 0.78         | 16.2% | −14.2% | 53.4%    | 118       | 1    |
| LSTM            | 0.65         | 13.1% | −18.7% | 51.2%    | 195       | 3    |

Table 3: Gross Performance Metrics (Before Transaction Costs)

**Insight:** What the Gross Results Suggest (Wrongly): On gross metrics, the narrative is clear: machine learning wins. XGBoost achieves the highest Sharpe ratio (0.78), the highest CAGR (16.2%), and the lowest maximum drawdown (−14.2%). A student or researcher presenting only these numbers would conclude that ML dominates classical strategies. This is the exact conclusion found in hundreds of papers and thousands of blog posts. It is also, as we will show, dangerously incomplete.

## 5.2 Net Performance (After Costs)

Table 4 presents the same strategies evaluated after deducting realistic transaction costs. We use a round-trip cost of 0.14% per trade (0.07% one-way), which includes brokerage, STT, stamp duty, and estimated slippage.

| Strategy        | Net Sharpe | CAGR  | Max DD | Total Costs | Rank | Δ Rank |
|-----------------|------------|-------|--------|-------------|------|--------|
| Buy-and-Hold    | 0.44       | 11.9% | −38.2% | 0.28%       | 3    | +2     |
| SMA Crossover   | 0.48       | 8.2%  | −24.6% | 2.10%       | 1    | +3     |
| Bollinger Bands | 0.42       | 7.1%  | −19.8% | 3.92%       | 4    | −1     |
| Random Forest   | 0.31       | 8.4%  | −16.4% | 11.48%      | 5    | −3     |
| XGBoost         | 0.19       | 6.8%  | −14.2% | 16.52%      | 6    | −5     |
| LSTM            | 0.08       | 3.2%  | −18.7% | 27.30%      | 6    | −3     |

Table 4: Net Performance Metrics (After Transaction Costs)

**Insight:** The Inversion: The rankings invert completely. XGBoost, the gross winner, becomes the net loser — its Sharpe drops from 0.78 to 0.19, a decline of 76%. The Simple Moving Average crossover, ranked 4th on gross metrics, becomes the net winner with a Sharpe of 0.48. The LSTM, with 195 trades per year, bleeds 27.3% of its returns to costs and achieves a net Sharpe of only 0.08.

### 5.3 The Complexity Penalty

Table 5 presents the Complexity Penalty for each strategy — the difference between gross and net Sharpe ratios. This metric quantifies the cost of sophistication.

| Strategy        | Gross Sharpe | Net Sharpe | Penalty | Penalty % | Robustness    |
|-----------------|--------------|------------|---------|-----------|---------------|
| Buy-and-Hold    | 0.45         | 0.44       | 0.01    | 2.2%      | Very High     |
| SMA Crossover   | 0.52         | 0.48       | 0.04    | 7.7%      | High          |
| Bollinger Bands | 0.58         | 0.42       | 0.16    | 27.6%     | Medium        |
| Random Forest   | 0.71         | 0.31       | 0.40    | 56.3%     | Low           |
| XGBoost         | 0.78         | 0.19       | 0.59    | 75.6%     | Very Low      |
| LSTM            | 0.65         | 0.08       | 0.57    | 87.7%     | Extremely Low |

Table 5: The Complexity Penalty by Strategy

The Complexity Penalty reveals a striking pattern: it increases monotonically with model complexity. The simplest strategy (Buy-and-Hold) loses 2.2% of its performance to costs. The most complex (LSTM) loses 87.7%. The relationship is not linear — it accelerates as trading frequency increases, suggesting a threshold effect beyond which additional model sophistication becomes actively harmful.

### 5.4 Sensitivity Analysis: Cost Variation

To test the robustness of our findings, we recompute net Sharpe ratios at four cost levels: 0.05%, 0.10%, 0.14% (baseline), and 0.20% round-trip per trade.

| Strategy        | 0.05% Cost | 0.10% Cost | 0.14% Cost | 0.20% Cost | Break-Even |
|-----------------|------------|------------|------------|------------|------------|
| Buy-and-Hold    | 0.44       | 0.44       | 0.44       | 0.43       | ~2.0%      |
| SMA Crossover   | 0.50       | 0.49       | 0.48       | 0.46       | 0.35%      |
| Bollinger Bands | 0.50       | 0.46       | 0.42       | 0.36       | 0.22%      |
| Random Forest   | 0.46       | 0.38       | 0.31       | 0.22       | 0.12%      |
| XGBoost         | 0.39       | 0.28       | 0.19       | 0.07       | 0.08%      |
| LSTM            | 0.25       | 0.16       | 0.08       | -0.03      | 0.05%      |

Table 6: Net Sharpe Ratios Under Alternative Cost Assumptions

**Insight:** Break-Even Insight: For the SMA crossover, the break-even cost is 0.35% — meaning the strategy remains profitable even if costs triple. For the LSTM, the break-even is 0.05% — a cost level achievable only by institutional traders with direct market access. At 0.20% costs, the LSTM generates negative risk-adjusted returns.

## 5.5 Statistical Significance

We test whether the difference in net Sharpe ratios between the SMA crossover and other strategies is statistically significant using a bootstrap procedure with 10,000 resamples.

| Comparison            | $\Delta$ Sharpe | 95% CI         | p-value | Significant? |
|-----------------------|-----------------|----------------|---------|--------------|
| SMA vs. Buy-and-Hold  | +0.04           | [−0.08, +0.16] | 0.42    | No           |
| SMA vs. Bollinger     | +0.06           | [−0.04, +0.18] | 0.31    | No           |
| SMA vs. Random Forest | +0.17           | [+0.05, +0.29] | 0.02    | Yes          |
| SMA vs. XGBoost       | +0.29           | [+0.14, +0.44] | 0.001   | Yes          |
| SMA vs. LSTM          | +0.40           | [+0.22, +0.58] | <0.001  | Yes          |

Table 7: Bootstrap Tests of Net Sharpe Ratio Differences (SMA Crossover as Benchmark)

The SMA crossover significantly outperforms all three machine learning strategies (Random Forest, XGBoost, LSTM) at the 5% level, with the advantage over XGBoost and LSTM being highly significant ( $p < 0.001$ ). This confirms that the inversion is not merely a numerical artifact but a robust statistical phenomenon.

## 6. Discussion

### 6.1 Why Complexity Fails

Our results challenge the prevailing assumption — reinforced by both academic research and industry marketing — that more sophisticated models necessarily yield better trading performance. The mechanism behind our findings is not model failure in the traditional sense (overfitting, data leakage, or poor architecture). Rather, it is a structural mismatch between model complexity and the cost structure of the market.

Machine learning models, particularly gradient-boosted trees and neural networks, are designed to maximize predictive accuracy. In the context of daily return forecasting, this means identifying subtle patterns — slight deviations from randomness — that might predict next-day direction with 52–54% accuracy. This is a genuine edge, albeit a small one. The problem arises when the model acts on this edge: it generates trading signals on a substantial fraction of days (80–120+ per year for XGBoost, nearly 200 for LSTM), and each signal incurs a fixed transaction cost.

The arithmetic is unforgiving. If a strategy identifies an edge of 0.05% per trade (a realistic estimate for daily directional forecasting on a broad index), but each trade costs 0.14%, the net expected return per trade is –0.09%. The model is not wrong about the direction; it is wrong about the magnitude of the edge relative to the cost of acting on it.

**Insight:** The Signal-to-Noise vs. Cost Arithmetic: Consider a strategy with 53% directional accuracy. On 100 trades, it wins 53 times and loses 47 times. If the average win is +0.8% and the average loss is –0.7%, the gross expected profit per trade is  $(0.53 \times 0.8\%) + (0.47 \times -0.7\%) = +0.095\%$ . After a 0.14% cost, the net is –0.045% per trade. The strategy is profitable in theory and bankrupt in practice. This is the Complexity Trap in arithmetic form.

## 6.2 The Role of Trading Frequency

Trading frequency emerges from our analysis as the single most important determinant of net strategy performance — more important than model architecture, feature engineering, or hyperparameter tuning. The relationship between frequency and net Sharpe is not merely negative; it is non-linear and exhibits a threshold effect. Below approximately 30 trades per year, the Complexity Penalty remains modest (under 15%). Above 80 trades per year, the penalty accelerates dramatically, exceeding 50% for Random Forest and 75% for XGBoost.

For researchers, this suggests that any backtest reporting gross performance without specifying trading frequency is incomplete and potentially misleading. For practitioners, particularly retail traders in emerging markets like India, the implication is that the pursuit of model sophistication may be counterproductive.

### **6.3 Market Microstructure and Emerging Markets**

Our findings are specific to the Indian equity market, where transaction costs are higher than in developed markets. In the United States, a retail trader using a zero-commission broker might face round-trip costs of 0.02–0.03%. In India, the Securities Transaction Tax (STT) alone is 0.1% on the sell side for equity delivery. The total cost structure is 3–5× higher than in the U.S., which amplifies the Complexity Trap.

However, we argue that the qualitative finding — that complexity increases vulnerability to costs — is generalizable across markets. Even in low-cost environments, the relationship between trading frequency and net performance is negative; the threshold simply shifts to higher frequencies. The Complexity Trap is a universal feature of friction-filled markets.

### **6.4 Limitations**

This study has several limitations. First, our cost model is simplified. We assume a fixed cost per trade, whereas in reality, costs vary with market volatility, liquidity, and order size. Second, our strategies are evaluated on a single asset — the NIFTY 50 index. Third, our machine learning models are relatively simple by industry standards. Production-grade quant funds use ensembles of models, alternative data sources, and sophisticated execution algorithms. Fourth, we do not account for taxes on capital gains.

### **6.5 Implications for Education and Practice**

The most consequential implication of our findings is pedagogical. Quantitative finance education overwhelmingly emphasizes model sophistication. Students are taught to build Random Forests, XGBoost models, and neural networks before they are taught to evaluate costs, model slippage, or audit backtests for leakage. The result is a generation of aspiring quants who can build impressive models but cannot distinguish a profitable strategy from a "paper tiger."

We propose a fundamental reordering of the quant curriculum: cost-aware evaluation before model complexity. A student who can build a moving average crossover, evaluate it with realistic costs, and identify its break-even cost threshold has learned more than is

practically useful than a student who has built an LSTM but never computed a net Sharpe ratio.

## **7. Conclusion**

This study presents a comprehensive, cost-aware comparison of classical and machine learning trading strategies on the NIFTY 50 index. Our central finding is that model sophistication and real-world profitability are inversely related after transaction costs — a phenomenon we term the Complexity Trap. While machine learning models achieve superior gross performance, their elevated trading frequency erodes this advantage and, in many cases, renders them inferior to simple classical strategies.

The Simple Moving Average crossover strategy, despite its minimal complexity, achieves the highest net Sharpe ratio (0.48) and demonstrates the greatest robustness to cost variation. The LSTM neural network, the most complex model evaluated, achieves the lowest net Sharpe (0.08) and becomes unprofitable at realistic retail cost levels. The Complexity Penalty — the difference between gross and net Sharpe — increases monotonically with model complexity, reaching 87.7% for the LSTM.

These findings challenge the prevailing narrative in quantitative finance education and popular discourse, which equates model sophistication with trading success. We argue that this narrative is not merely incomplete but actively harmful, as it encourages students and retail traders to invest in complex models without understanding the cost structure that determines real-world profitability.

Our proposed solution is straightforward: mandatory cost-aware evaluation. Every backtest should report net as well as gross performance. Every strategy comparison should include trading frequency. Every student project should compute a break-even cost threshold. The Complexity Penalty metric provides a simple, interpretable framework for implementing this standard.

Future research should extend this analysis to individual stocks, intraday data, and alternative asset classes. It should also incorporate more sophisticated cost models that capture volatility-dependent slippage and market impact.

*Before you build a neural network, build a cost model. Before you optimize for accuracy, optimize for net Sharpe. And before you celebrate a gross backtest, ask what remains after the market takes its cut.*

## References

- Brock, W., Lakonishok, J., & LeBaron, B. (1992). Simple technical trading rules and the stochastic properties of stock returns. *Journal of Finance*, 47(5), 1731–1764.
- Fischer, T. G., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654–669.
- Gu, S., Kelly, B., & Xiu, D. (2020). Empirical asset pricing via machine learning. *Review of Financial Studies*, 33(5), 2223–2273.
- Han, Y., Yang, K., & Zhou, G. (2016). A new anomaly: The cross-sectional profitability of technical analysis. *Journal of Financial and Quantitative Analysis*, 51(5), 1857–1882.
- Kaufman, P. J. (2013). *Trading Systems and Methods* (5th ed.). Wiley.
- López de Prado, M. (2018). *Advances in Financial Machine Learning*. Wiley.
- Shah, A., & Thomas, S. (2019). *Securities Markets: Principles and Practice*. Tata McGraw-Hill.
- Zhang, S., Wang, D., & Zhang, R. (2021). Machine learning for cross-sectional stock returns: A comparison of methods. *Journal of Banking & Finance*, 127, 106–124.

## Appendix A: Complete Reproducible Code

The following Python script reproduces the complete empirical analysis presented in this paper. It uses standard open-source libraries including pandas, numpy, matplotlib, scikit-learn, xgboost, tensorflow, and yfinance. Running the script in Google Colab or Jupyter Notebook generates the results, tables, and figures discussed in the main text.

=====

### Complete Analysis Script (Google Colab):

[<https://colab.research.google.com/drive/1UhUv8jnmNNjNNyQY3V-zZtfXEWK4ZEW7?usp=sharing>]

=====

### **Important Note for Reproducibility**

The Random Forest, XGBoost, and LSTM strategies require the complete training pipelines described in Sections 4.7–4.9. The script provided here demonstrates the core framework and classical strategies. For the full implementation, including feature engineering, model training, and evaluation of all six strategies, refer to the accompanying GitHub repository.

### **Appendix B: Leakage Audit Checklist**

Every backtest in this study was subjected to the following leakage audit before any results were reported:

0. Target Verification: Confirm that the target variable (next-day return) uses only `.shift(-1)` and no overlapping windows.
1. Feature Verification: Confirm that every feature is computed using only data available at or before the trading decision time.
2. Train-Test Split: Confirm chronological ordering (no shuffle). Verify that the test period is strictly after the training period.
3. Scaling Verification: Confirm that `StandardScaler` is fitted on training data only.
4. Signal Lag: Confirm that all trading signals are shifted by one day (`.shift(1)`) to prevent using today's information to trade today.

All five checks passed for every strategy reported in this paper. The full audit log is available in the GitHub repository.

# The Complexity Trap

*Sachin Kumar · IIT Madras · 2026*

[github.com/sachin23-an](https://github.com/sachin23-an)